```
// include
using namespace cute;
// ready to optimize
```

CuTe Layouts and Layout Algebra

The Missing Manual

How NVIDIA's CuTe abstraction tames the complexity of multi-dimensional memory access on GPUs.

Layout Algebra — Cris Cecka.
Slides — Jean-Luc Duprat, 2026.

Row-Major vs Column-Major: Two Ways to Linearize

Matrix Size: (M, N) — M Rows, N Columns

Row-Major (C/C++)

0	1	2	3
4	5	6	7
8	9	10	11

offset = row * N + col

Cell (1, 2): 1 * 4 + 2 = 6
(Uses N = 4 Columns)

Column-Major (Fortran/MATLAB)

0	3	6	9
1	4	7	10
2	5	8	11

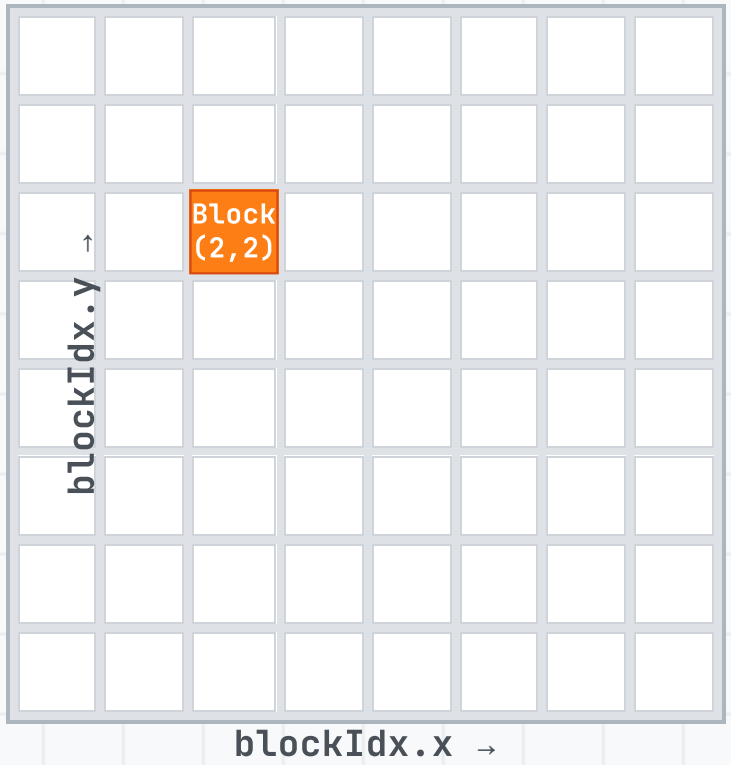
offset = col * M + row

Cell (1, 2): 2 * 3 + 1 = 7
(Uses M = 3 Rows)

Same data, same cell — **different offset**. The layout choice is baked into every index expression.

Block Tiles (Grid Level)

Partitioning the Global Matrix into Block Tiles



1. The Grid

The entire matrix is divided into independent **Block Tiles** processed by Thread Blocks.

2. Block Coordinates

Each Block knows its position via blockIdx.

```
Row = blockIdx.y * BM + ...
Col = blockIdx.x * BN + ...
```

Scale

Typical Block Tile Size (BM x BN): 128x128 or 64x64 elements.

First level of hierarchy: Selecting the Block Tile.

Visualizing Warp Tiles & Thread Tiles

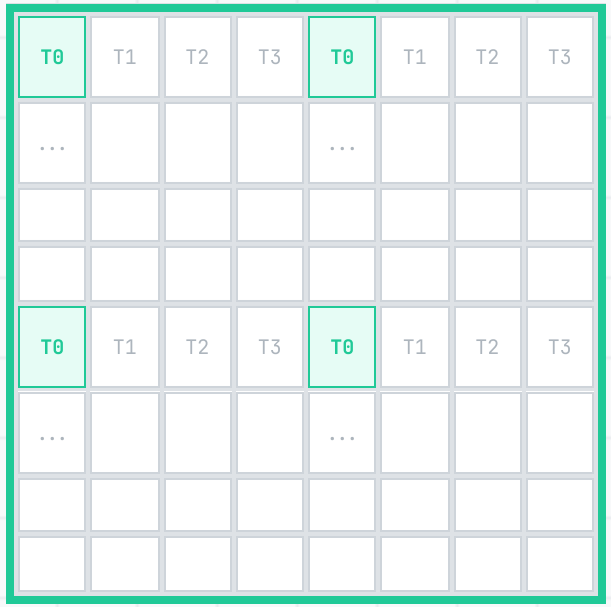
Inside the Block: Subdividing into Warp Tiles and Values

1. Inside Block Tile

Block partitioned into Warp Tiles

→

2. Inside Warp Tile



Thread Tile: T0 owns 4 distributed values

```
Offset = Block_Base + Warp_Offset + Thread_Offset
```

*Second level: Distributing the Tile to Threads (each thread owns a **Thread Tile**).*

With Tiling Indexing Arithmetic Explodes

Each level of tiling multiplies the number of index variables.

Simple Access

```
offset = row * N + col
```

2 vars

↓ *Add Block Tile (blockIdx selects tile, threadIdx selects element)*

Block Tile

```
offset = (blockIdx.y * BM + threadIdx.y) * N + (blockIdx.x * BN +
threadIdx.x)
```

6

↓ *Add Warp Tile (warpIdx selects Warp Tile, thread iterates within)*

Warp Tile

```
offset = (blockIdx.y*BM + warpRow*WM + tRow*TM + i) * N + ...
```

8+

Every tiling level adds **2+ new variables** and **new multiplication terms** to every index expression.

Anatomy of the Nightmare: Where Does Each Term Come From?

Color-coding reveals 3 tangled levels of tiling in a single index expression.

REGISTER ACCUMULATOR INDEX

```
threadResults[(warpRow * WM + resIdxM) * (WNITER * TN) +
              (warpCol * TN) + resIdxN]
```

SHARED MEMORY LOAD INDEX

```
As[(dotIdx * BM) + warpRow * WM +
   threadRowInWarp * TM + i]
```

GLOBAL MEMORY STORE INDEX

```
C[(blockIdx.y*BM + warpRow*WM + threadRowInWarp*TM + resIdxM) * N +
  (blockIdx.x*BN + warpCol*WN + threadColInWarp*TN + resIdxN)]
```

Tiling Hierarchy

- Block / CTA Level
- Warp Level
- Thread Level
- Accumulation Loop

Variable Decoder

BM/BN	Block Tile Size
WM/WN	Warp Tile Size
TM/TN	Thread Tile Size
resIdx	Thread Index
dotIdx	K-Loop Index

3 tiling levels × 2 dimensions = 8+ variables tangled into every single index expression.

The Problem: Maintenance Nightmare

GPU tensor indexing without abstractions is a sea of integer arithmetic.

raw_cpp_nightmare.cu

```
// WNITER: Iterations per warp
// Warptiling index calculation
threadResults[(warpRow * WM + threadValM) * (WNITER * TN) +
              (warpCol * TN) + threadValN] +=
    regM[warpRow * WM + threadValM] *
    regN[warpCol * TN + threadValN];

// Shared memory loading
As[(dotIdx * BM) + warpRow * WM +
   threadRowInWarp * TM + i];
```

Questions every programmer must answer:

> What is the shape of As? Which dimension is contiguous?

> How does threadRowInWarp relate to warpRow?

> What happens if you change BM? (A single off-by-one error fails silently)

This is the status quo. CuTe eliminates it entirely.

What is CuTe?

CuDA Template utilities

NVIDIA's C++ template library for expressing tensor layouts and operations with mathematical precision.

Core abstraction underpinning CUTLASS 3.x.

> THE CENTRAL INSIGHT
>
> A Layout is a function from coordinates to memory indices.

Once represented as a function, layouts can be composed, divided, tiled, and transformed algebraically.

Key Capabilities

- A `Layout` type that encodes shape and stride as a single object
- An algebra of layouts: composition, complement, division, and product
- `Tensor` objects that pair a Layout with a data pointer
- Partitioning utilities (`local_tile`) that express tiling in one line
- Static and dynamic integers treated uniformly for optimization

Integers in CuTe: Static vs. Dynamic

Uniform treatment enabling powerful static optimizations

Dynamic Integers

`int, size_t`

Ordinary variables known only at run time. Used when dimensions depend on user input.

```
// Shape known at run time
auto layout = make_layout(
  make_shape(M, K)
);
// (M,K):(1,M)
```

Static Integers

`cute::C<N>, _1, _2`

Value is encoded in the **type** itself. Math on static integers produces static integers.

```
// Everything known at compile time
auto layout = make_layout(
make_shape(_128{}, _8{})
);
// (_128,_8):(_1,_128)
```

WHY STATIC MATTERS: COMPILER SUPERPOWERS

- ✓ Eliminate runtime index arithmetic
- ✓ Statically allocate shared buffers
- ✓ Verify layout compatibility
- ✓ Dispatch to optimized vector instructions

Layouts: Shape and Stride

Layout = (Shape, Stride) = Shape:Stride

Shape (IntTuple)

Defines the **coordinate space**.
How many elements exist in each dimension?

Stride (IntTuple)

Defines the **memory mapping**.
How many elements to skip per step in each dimension?

INDEXING

$$\text{offset} = \text{coord} \cdot \text{stride} = \sum (x_i \times s_i)$$
$$= x_0 * \text{stride}_0 + x_1 * \text{stride}_1 + \ldots + x_n * \text{stride}_n$$

ROW-MAJOR 4X8 MATRIX

```
auto layout = make_layout(
make_shape(4, 8),
make_stride(8, 1));

// (4,8):(8,1)
```

COLUMN-MAJOR 4X8 MATRIX

```
auto layout = make_layout(
make_shape(4, 8),
make_stride(1, 4));

// (4,8):(1,4)
```

Visualizing Simple Layouts

Seeing the index mapping concretely builds intuition

Row-Major

(2,4):(4,1)

	c0	c1	c2	c3
r0	0	1	2	3
r1	4	5	6	7

Contiguous in memory along rows

Column-Major

(2,4):(1,2)

	c0	c1	c2	c3
r0	0	2	4	6
r1	1	3	5	7

Contiguous in memory along cols

Stride-2

(2,4):(4,2)

	c0	c1	c2	c3
r0	0	2	4	6
r1	4	6	8	10

Skips elements (e.g. 1, 3, 5...)

Algorithms work the same on all layouts.

The 1-D Coordinate Space

Layouts as Functions on Integers

A key CuTe insight: every multi-dimensional layout also defines a **1-D coordinate space**.

A single integer **i** from 0 to size(layout)-1 maps to coordinates in **colexicographic** (column-major) order.

Layouts are functions from integers to integers.

```
// Generic algorithm works for ANY layout
for (int i = 0; i < size(layout); ++i) {
    process(data[layout(i)]);
}
```

Example Layout: Shape (2, 4)

1-D Index	2-D Coord	Offset (Col-Major)
0	(0,0)	0
1	(1,0)	1
2	(0,1)	2
3	(1,1)	3
4	(0,2)	4
5	(1,2)	5
6	(0,3)	6
7	(1,3)	7

Hierarchical Layouts

Layouts can be nested to represent tiling (e.g.: match hardware hierarchy)

The Core Concept

In CuTe, a "Shape" is not just a list of integers.

A Shape can contain **other Shapes**.
A Coordinate matches this structure.

Shape: (A, (B, C))

⬇ matches ⬇

Coord: (a, (b, c))

3 Ways to Address the Same Data:

1. Full	`Layout( a, (b, c))`	*Matches the hierarchy exactly.*
2. Mixed	`Layout( a, i )`	*Flattens the inner tuple (B,C) → i.*
3. Flat	`Layout( idx )`	*Treats the entire tensor as 1D.*

Why? This allows us to group dimensions (e.g., a **Tile**) while keeping them logically connected, yet address them flexibly depending on the algorithm's needs.

Why Hierarchical Shapes?

Encoding multi-level tiling in a single Layout object

The Problem with Flat Shapes

GPUs use multiple tiling levels (CTA, warp, thread, ...). A flat shape like `(M, K)` cannot express those nested tiles, forcing manual boundary logic.

The Hierarchical Solution

A nested shape like `(_2, (_2,_2))` encodes tile selection and intra-tile layout together, letting composable ops add or remove nesting.

Concrete Example

```
// Shape: (_2, (_2, _2))
// Stride: (_4, (_2, _1))
auto layout = make_layout(
      make_shape(_2{},  make_shape(_2{},_2{})),
     make_stride(_4{}, make_stride(_2{},_1{})));
```

Outer mode (_2:_4)	Selects between 2 tiles, spaced by the outer stride.
Inner mode ((_2,_2):(_2,_1))	A 2x2 sub-tile (row-major) inside each tile.

Key insight: Each nesting level mirrors a tiling level; strides show physical spacing.

1. **Read inside-out.** Innermost tuple = finest granularity.
2. **Match levels to hardware.** Outer = CTA, middle = warp, inner = thread.
3. **Strides tell spacing.** Each level's stride shows memory distance.

CuTe Layout Operations

Foundational operations for inspecting and manipulating layouts

Operation	Description	Example
rank(L)	Number of top-level modes	`rank((4,8):(1,8)) = 2`
size(L)	Total number of elements	`size((4,8):(1,8)) = 32`
shape(L)	The shape IntTuple	`shape((4,8):(1,8)) = (4,8)`
stride(L)	The stride IntTuple	`stride((4,8):(1,8)) = (1,8)`
cosize(L)	Size of the codomain	`cosize((4,8):(2,1)) = 25`
get<I>(L)	The I-th sub-layout	`get<0>((4,8):(1,8)) = 4:1`
select<I...>(L)	Select specific modes	`select<0,2>((2,3,5):(1,2,6))`
flatten(L)	Remove all nesting	`flatten((2,(3,4)):...) = (2,3,4):...`
coalesce(L)	Simplify adjacent modes	`coalesce((_2,_4):(_1,_2)) = _8:_1`

These operations form the foundation of the Layout Algebra.

Layout Algebra

Transforming layouts with function algebra

Why Algebra?

Manual indexing is fragile and hard to verify. **Algebra is composable and provable.** Instead of writing loops, we compose functions.

The Function Analogy

Since a Layout is a function $L: \mathbb{Z}^n \to \mathbb{Z}$, we can apply classical function operations:

- **Reshaping** = Composition ($f \circ g$)
- **Tiling** = Domain Restriction
- **Partitioning** = Domain Division

→ **Coalesce**
Simplify representation (merge adjacent modes)

∘ **Composition**
Chain functions: $R(c) = A(B(c))$

* **Complement**
Find the "rest" of the domain ($A*$)

⊘ **Division**
Split into (Tile, Rest): $A \circ (B, B*)$

⊗ **Product**
Replicate Tile over Rest: $(A, A* \circ B)$

We will now dive deep into each of these operations to build intuition.

Coalesce

Simplifying representation without changing semantics

Why Coalesce?

Tiling often creates redundant modes (e.g., size-1 dimensions). Coalesce cleans them up, like reducing a fraction (4/8 → 1/2).

```
// A messy layout from tiling:
auto layout = make_layout(
  make_shape (_2, _2, _2, _2),
  make_stride(_1, _2, _4, _8));
```

(1:d, s:d) → s:d

Drop size-1 modes (identity)

(s0:d0, s1:(s0*d0)) → (s0*s1):d0

Merge when stride matches (contiguous)

Otherwise → Keep Separate

Preserve structure if strides don't align

↓ coalesce(layout)

```
// Simplified result:
auto result = _16:_1;
// Just a contiguous vector!
```

*Key Insight: Coalesce changes the **Layout object**, but the **index mapping** stays exactly the same.*

Composition

Chaining functions to create views

$$R = A \circ B$$

$R(c) = A(B(c))$

"Apply layout B, then layout A"

Coordinate c → Logical Index (Layout B) → Physical Offset (Layout A)

```
// A: 10 elements, stride 1
auto A = 10:1;

// B: Select 5 elements, stride 2
auto B = 5:2;
```

```
// R = A ∘ B
// "Select every 2nd element of A"
auto R = 5:2;
// Result has stride 2!
```

Intuition: B is a "View" into A. R describes the View in terms of physical memory.

Composition: Worked Example

Reshaping a 1D array into a 2D matrix

```
// A: 1D array, stride 2
auto A = 20:2;

// B: 5x4 row-major view
auto B = (5,4):(4,1);

// Compose!
auto R = composition(A, B);
```

Tracing coordinate (1, 2):

1. Apply B (View):
   `B(1,2) = 1*4 + 2*1 = 6`
2. Apply A (Data):
   `A(6) = 6*2 = 12`
3. Result (Direct):
   `R(1,2) = 1*8 + 2*2 = 12`

R = compose(A,B) = (5,4):(8,2)

Complement

The algebra of "what remains"

> If layout **A** selects a subset of indices in range [0, M), complement(A, M) describes the layout of the **remaining indices**.

A = 4:1 (Selects 0,1,2,3)

A* = 6:4 (Selects 4,8,12...)

0	1	2	3	4	5	6	7	8	9	10	11	12	...

(Visual simplified: A* actually selects the *starts* of the next blocks)

Key Insight: If A is the "Tile", then A* is the "Grid of Tiles".

Complement: Strided Example

Filling in the gaps

```
complement(4:2, 24) = (2, 3):(1, 8)
```

Why (2, 3):(1, 8)?

Inner (2:1): Fills the gap of size 1 between the stride-2 elements (at offsets 1, 3, 5, 7).
Outer (3:8): Jumps to the next block of 8 (since 4*2=8 covers the first block).
Together, they cover all remaining 20 elements.

This operation is automatic. You don't derive it by hand.

Index Decomposition: Column-Major

From Index to Coordinate to Offset

Layout: (_4, _6) : (_1, _4)

Colexicographical Order
(First Mode Varies Fastest)

0	4	8	12	16	20
1	5	9	**13**	17	21
2	6	10	14	18	22
3	7	11	15	19	23

STEP 1: DECOMPOSITION (VIA SHAPE)

Index: 13
Row (13 % 4): 1
Col (13 / 4): 3
Coordinate: (1, 3)

STEP 2: OFFSET CALCULATION (VIA STRIDE)

Row Offset: 1 × _1 = 1
Col Offset: 3 × _4 = 12
Total Offset: 1 + 12 = 13

*Note: Decomposition depends **only on Shape**. Physical location depends on **Stride**.*

Index Decomposition: Row-Major

Same Shape (4, 6), Different Layout

Layout: (_4, _6) : (_6, _1)

Lexicographical Order
(Last Mode Varies Fastest)

STEP 1: DECOMPOSITION (VIA SHAPE)

Index:	13
Row (13 / 6):	2
Col (13 % 6):	1
Coordinate: (2, 1)	

STEP 2: OFFSET CALCULATION (VIA STRIDE)

Row Offset:	2 × _6 = 12
Col Offset:	1 × _1 = 1

Total Offset: 12 + 1 = 13

*Notice: In Row-Major, we divide by **Cols** (6) to get Row, and mod by **Cols** (6) to get Col.*

Hierarchical Decomposition

The Shape Dictates the Divisors

Shape: (_2, (_3, _4))

Step 1: Process First Mode

$\text{Coord}_0 = 13 \% 2 = 1$

$\text{Rest} = 13 / 2 = 6$

Peel off the first mode (Size 2). The quotient (6) passes to the next step.

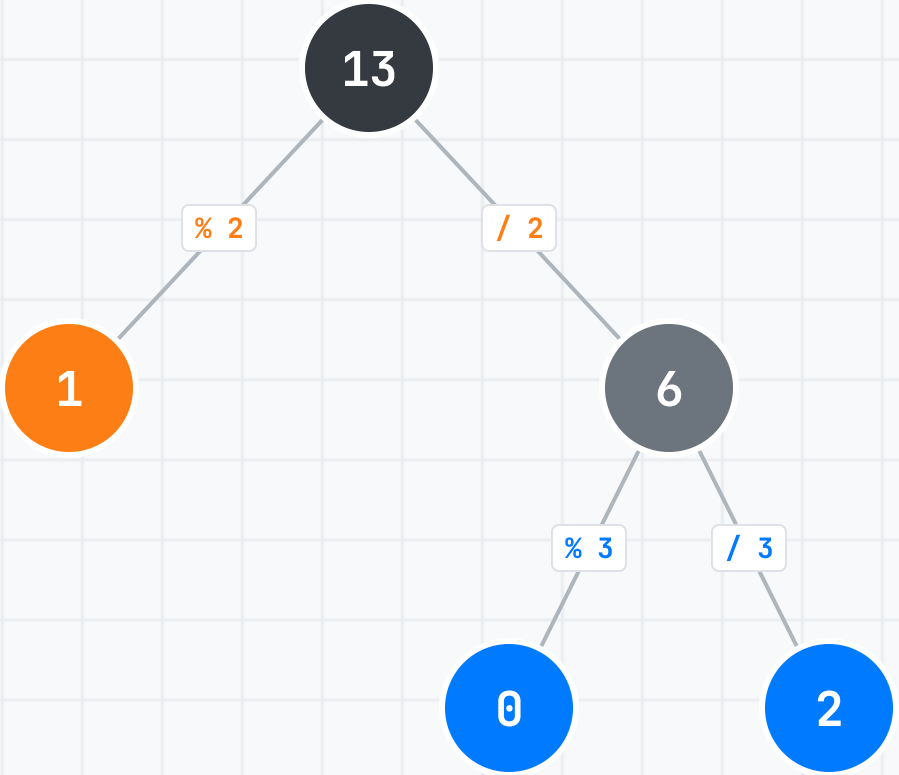
Step 2: Process Next Mode

$\text{Coord}_{1a} = 6 \% 3 = 0$

$\text{Coord}_{1b} = 6 / 3 = 2$

Peel off the nested mode (Size 3).

The **Mod/Div factor** is simply the size of the current mode.



Result: (1, (0, 2))

Generalized Decomposition

When Strides Break the Order

The Challenge: Row-Major

```
Layout: (_4, _6) : (_6, _1)
```

Standard Left-to-Right peeling fails here because the **First Mode** (Size 4) has the **Largest Stride** (6).

If we peel Size 4 first:
Index 13 % 4 = 1 (Row)
Index 13 / 4 = 3 (Col)
Result: (1, 3) → Wrong!

The Solution: Follow Strides

1. Identify **Smallest Stride** (Fastest Mode). Stride `_1` corresponds to **Mode 1 (Size 6)**.
2. Peel **Mode 1** first!

```
Col = 13 % 6 = 1
Rest = 13 / 6 = 2
```

3. Peel **Next Smallest** (Mode 0, Size 4).

```
Row = 2 % 4 = 2
```

Correct Result: (2, 1)

The General Algorithm:
Always peel modes in order of **Increasing Stride.**

(Smallest Stride = Fastest Mode = Peel First)

Division

Splitting a layout into "Tile" and "Rest"

$$A \oslash B := A \circ (B, B^*)$$

Result Shape: ((Tile), (Rest))

Original (12 elements):

0	1	2	3	4	5	6	7	8	9	10	11

↓ Divide by Tile(4)

Result (3 Tiles of 4):

Rest=0				Rest=1				Rest=2			
0	1	2	3	4	5	6	7	8	9	10	11

Coordinate: (Inner, Outer) Inner: Offset within Tile Outer: Which Tile?

Division: Three Variants Illustrated

Same Data, Different Structures

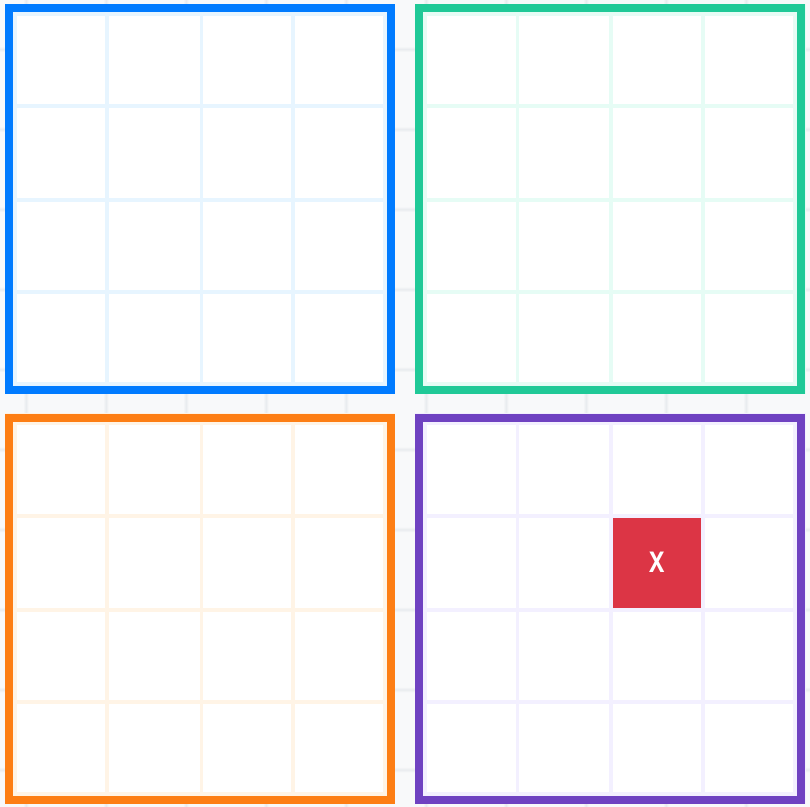
Input: (6, 4) ⊘ Tile: (3, 2)

VARIANT	RESULTING SHAPE	RATIONALE & USE CASE
logical_divide (zipped)	((3,2), (2,2))	STANDARD TILING **Preserves 2D structure.** Keeps Tile and Rest separate. Best when iterating over the "Grid" (Rest) while processing the "Tile" as a unit.
tiled_divide	((3,2), 2,2)	PARTIAL FLATTENING **Unpacks the Rest.** Tile is grouped, but Rest modes are appended. Useful for slicing specific Grid dimensions without full unpacking.
flat_divide	(3,2, 2,2)	REGISTER ARRAYS **Fully Flat.** Removes all nesting. Best for 1D register files or when hierarchical structure is no longer needed (e.g., after loading).

● **Inner (Tile):** The part you requested ● **Outer (Rest):** The remainder

Division: 2D Matrix Tiling Visualized

Coordinate Mapping for ((M, N), (m, n))



Result Shape

((4, 4), (2, 2))

((Tile_M, Tile_N), (Grid_M, Grid_N))

```
Global Coord: (5, 6)

Outer (Tile Idx):
  5/4 = 1, 6/4 = 1

Inner (Offset):
  5%4 = 1, 6%4 = 2

Result: ((1, 2), (1, 1))
```

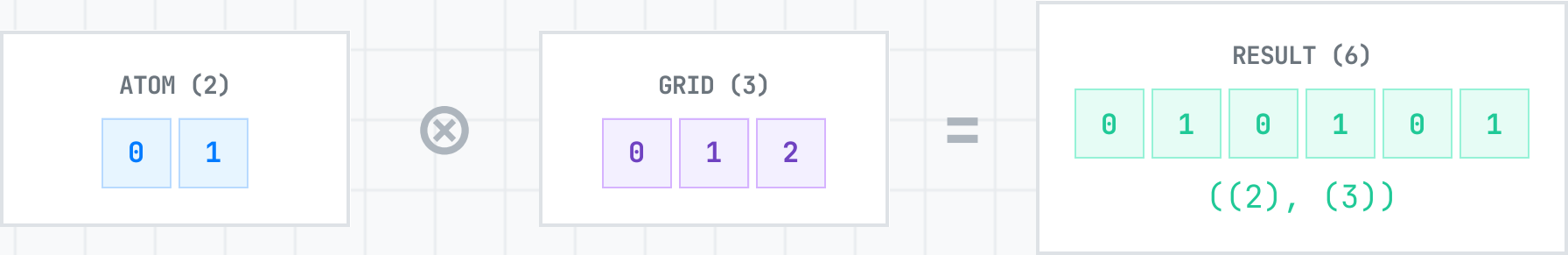
We just viewed linear memory as a 4D tensor. No data moved.

Product

Replicating a layout according to a pattern

$$A \otimes B := (A, A^* \circ B)$$

Result Shape: ((Atom), (Grid))



Conceptually: "Repeat Atom 3 times."

Product: Three Variants Illustrated

Replication Strategies

Atom: (2) Grid: (3)

VARIANT	SHAPE & PATTERN	RATIONALE & USE CASE
logical_product (zipped)	((2), (3)) Structure preserved	DEFAULT **Preserves Structure.** Keeps Atom and Grid separate. Use this when you want to maintain the logical distinction (e.g., Thread vs Value).
blocked_product	(2, 3) 0 1 0 1 0 1 Contiguous Atom	VECTORIZATION **Atom Inner (Stride 1).** The Atom is kept contiguous in memory. Essential for **vectorized loads/stores** (e.g., `ld.global.v4`) where threads must access adjacent elements.
raked_product	(3, 2) 0 0 0 1 1 1 Interleaved Atom	BANK CONFLICT AVOIDANCE **Grid Inner (Stride 1).** The Atom elements are spread out by `GridSize`. Useful for **cyclic distribution** to avoid shared memory bank conflicts or to maximize instruction-level parallelism.

"Blocked" keeps the Atom together. "Raked" spreads it out like a rake.

Product: Thread-to-Data Assignment

Blocked vs Raked Distribution

Threads: 8 Elements: 32 Per Thread: 4

Blocked (Contiguous Chunks) - Good for Vector Loads

T0	T0	T0	T0	T1	T1	T1	T1	T2	T2	T2	T2	T3	T3	T3	T3	T4	T4	T4	T4	T5	T5	T5	T5	T6	T6	T6	T6	T7	T7	T7	T7

Raked (Interleaved/Cyclic) - Good for Bank Conflict Avoidance

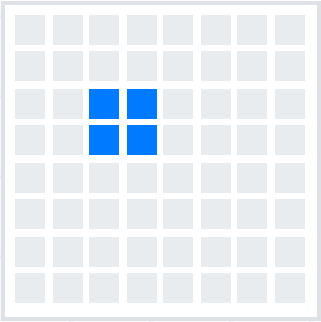
T0	T1	T2	T3	T4	T5	T6	T7	T0	T1	T2	T3	T4	T5	T6	T7	T0	T1	T2	T3	T4	T5	T6	T7	T0	T1	T2	T3	T4	T5	T6	T7

CuTe gives you precise control over this mapping with one function call.

Tiling in Practice: Full Visual Walkthrough

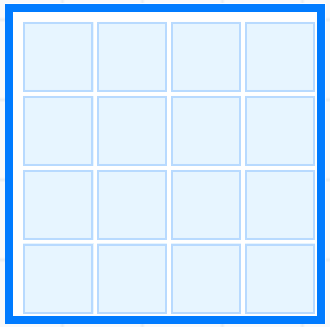
From Global Matrix to Thread Register

1. Global Tensor



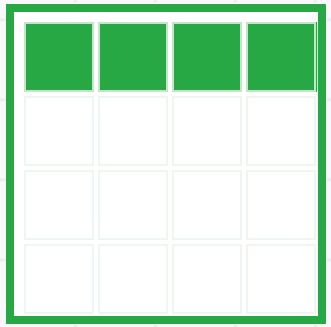
→

2. Block Tile



→

3. Thread Tile



```
// Global Tensor
Tensor gA = ...;
```

```
// Slice Block Tile
auto tA = local_tile(gA,
BlockTile, blk_coord);
```

```
// Partition to Thread
auto tA_t =
local_partition(tA, Layout,
tid);
```

This pattern (Global → Block → Thread) is universal in CuTe kernels.

Swizzling: Bank Conflicts Visualized

Naive Row-Major Layout (Stride = 8 Banks)

	B0	B1	B2	B3	B4	B5	B6	B7
T0	0	1	2	3	4	5	6	7
T1	0	1	2	3	4	5	6	7
T2	0	1	2	3	4	5	6	7
T3	0	1	2	3	4	5	6	7
T4	0	1	2	3	4	5	6	7
T5	0	1	2	3	4	5	6	7
T6	0	1	2	3	4	5	6	7
T7	0	1	2	3	4	5	6	7

The Scenario

Warp 0 (T0-T7, Blue) loads **Column B0**.
T0 reads Row 0, T1 reads Row 1, etc.

The Conflict

Look at the Red Box.
Every thread hits **Bank 0**.
8-way Serialization!

Vertical alignment of the same color = Bank Conflict.

(Note: Real hardware has 32 Banks & 32 Threads/Warp. Simplified to 8x8 for clarity.)

Swizzling: XOR Diagonalization Visualized

Bank = (Row XOR Col)

	B0	B1	B2	B3	B4	B5	B6	B7
T0	0	1	2	3	4	5	6	7
T1	1	0	3	2	5	4	7	6
T2	2	3	0	1	6	7	4	5
T3	3	2	1	0	7	6	5	4
T4	4	5	6	7	0	1	2	3
T5	5	4	7	6	1	0	3	2
T6	6	7	4	5	2	3	0	1
T7	7	6	5	4	3	2	1	0

The Magic

The same Column 0 access pattern.
But look at the **Green Box**.

Conflict Free

T0 reads B0, T1 reads B1, T2 reads B2...
Every thread hits a different bank!
100% Bandwidth.

Vertical alignment of DIFFERENT colors = Success.

(Note: Real hardware has 32 Banks & 32 Threads/Warp. Simplified to 8x8 for clarity.)

Decoding the Swizzle Template

Swizzle<B, M, S>

B: Base Bits

Number of LSBs to **preserve** (not swizzled). Typically matches the vector size (e.g., 3 bits = 8 elements).

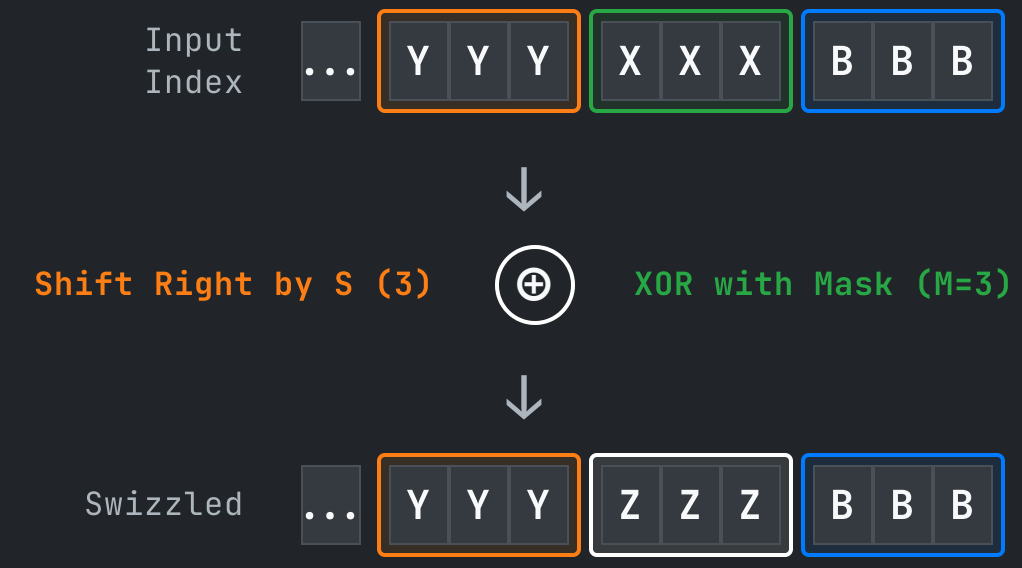
M: Mask Bits

Number of bits to **XOR**. Controls the "swizzle pattern width" (e.g., 3 bits = 8 banks).

S: Shift

Distance to shift upper bits right before XORing. Typically S = B (shift by vector width).

Example: Swizzle<3, 3, 3>



$$Z = X \oplus Y$$

(Target Bits) XOR (Upper Bits)

Layout Algebra: Complete Toolkit Summary

The 5 Operations You Need to Know

Operation	Formula	Purpose	Key Use Case
coalesce	L → L'	Merge redundant modes	Cleanup after tiling
composition	A ∘ B	Reshape / Slice / Reorder	Viewing data differently
complement	A*	Find the "Rest"	Basis for tiling
logical_divide	A ∘ (B, B*)	Split into (Tile, Grid)	CTA Tiling, Partitioning
logical_product	(A, A* ∘ B)	Replicate Atom over Grid	Building Hierarchies

Now we have the tools to build a GEMM kernel.

GEMM: The Problem Setup

Maximizing performance requires at least 3-level tiling hierarchy

LEVEL 1: CTA / THREAD BLOCK
Global Memory → Shared Memory
Tile: 128×128×8

↓

LEVEL 2: WARP
Shared Memory → Registers
Tile: 64×64×8

↓

LEVEL 3: THREAD / MMA
Register File → Tensor Core
Tile: 16×8×16

The Engineering Challenge

! **Data Movement**
We must slice the global tensor A into CTA tiles, then Warp tiles, then Thread fragments.

! **Coordinate Transformation**
A thread needs to know: "Which element of the Global Tensor do I own?"

! **Bank Conflicts**
Shared memory layouts must be swizzled to avoid conflicts.

In raw C++, managing these indices manually is error-prone and unreadable.

GEMM in Raw C++: Complexity Begins

Manual index arithmetic is fragile and hard to read

```
__global__ void
gemm_naive(float* A, float* B, float* C, int M, int N, int K)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; ++k)
            sum += A[row * K + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}
```

⚠ Hardcoded Layouts

Assumes Row-Major; switching layouts needs many edits.

⚠ Fragile Arithmetic

row * K + k is easy to mistype.

⚠ No Tiling Abstraction

No explicit Tile object; tiling is implicit.

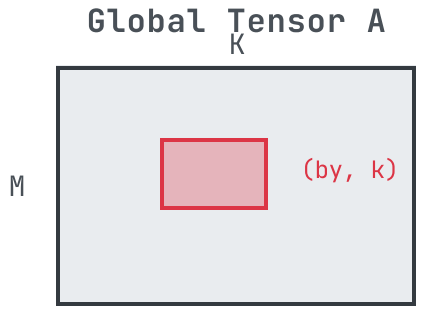
This is the simplest possible case. Real performance requires Shared Memory tiling...

GEMM in Raw C++: 2D Blocktiling

Manual coordinate calculations for every tile

Raw C++ Kernel

```
// 1. Calculate Global Offsets
int row_offset = blockIdx.y * BM;
int col_offset = blockIdx.x * BN;

// 2. Advance Pointers
const float* A_ptr = A + row_offset * K;
const float* B_ptr = B + col_offset;

// 3. Main Loop
for (int k = 0; k < K; k += BK) {
    // Load global → shared
    int tid = threadIdx.x;
    int load_row_a = tid / BK;
    int load_col_a = tid % BK;

    As[load_row_a][load_col_a] =
        A_ptr[load_row_a * K + load_col_a];

    __syncthreads();

    // Compute (math on regs)
}
```



⚠️ The Problem

We manually flatten 2D coordinates into 1D offsets.
Every access requires `row * stride + col` arithmetic.

GEMM in Raw C++: Warptiling Nightmare

When index arithmetic becomes unmaintainable

Implicit Thread Mapping?

```
late Warp-level offsets
= threadIdx.x % 32;
int warp_row = (lane / 4);
int warp_col = (lane % 4) * 2;

// Swizzled Shared Memory Access
int smem_offset =
  (warp_row * 64) +
  (warp_col ^ (warp_row / 8));

Fragment
al = smem_ptr[smem_offset];
```

Why XOR? Bank Conflicts?

Magic Numbers Everywhere!

This code has zero semantic meaning. It is purely structural

The Tensor Abstraction

Coupling Layouts with Data

Layout + **Pointer** = **Tensor**

```
// 1. Raw Data Pointer
float* data_ptr = ...;

// 2. Define Geometry
auto layout = make_layout(Shape<4,8>{}, Stride<8,1>{});

// 3. Create View
auto tensor = make_tensor(data_ptr, layout);

// 4. Access
tensor(2, 3) = 10.0f;
```

VIEW SEMANTICS

Lightweight handles: copying a tensor copies pointer + layout, not data.

UNIFIED MEMORY

Same behavior across Global, Shared, and Register memory.

HIERARCHICAL

Slice, tile, and partition tensors via the same algebraic ops.

GEMM in CuTe: Problem Setup

Replacing pointer arithmetic with Tensor objects

```
// 1. Shapes & Strides
auto M = int(m);
auto N = int(n);
auto K = int(k);

// 2. Global Tensors
// A: (M, K) Row-Major
auto gA = make_tensor(make_gmem_ptr(A),
                      make_layout(make_shape(M,K),
                      make_stride(K,_1{})));
// B: (N, K) Row-Major
auto gB = make_tensor(make_gmem_ptr(B),
                      make_layout(make_shape(N,K),
                      make_stride(K,_1{})));
// C: (M, N) Col-Major
auto gC = make_tensor(make_gmem_ptr(C),
                      make_layout(make_shape(M,N),
                      make_stride(_1{},M)));
```

A

Tensor gA

(M, K) : (K, 1)

Global Memory, Row Major

B

Tensor gB

(N, K) : (K, 1)

Global Memory, Row Major

C

Tensor gC

(M, N) : (1, M)

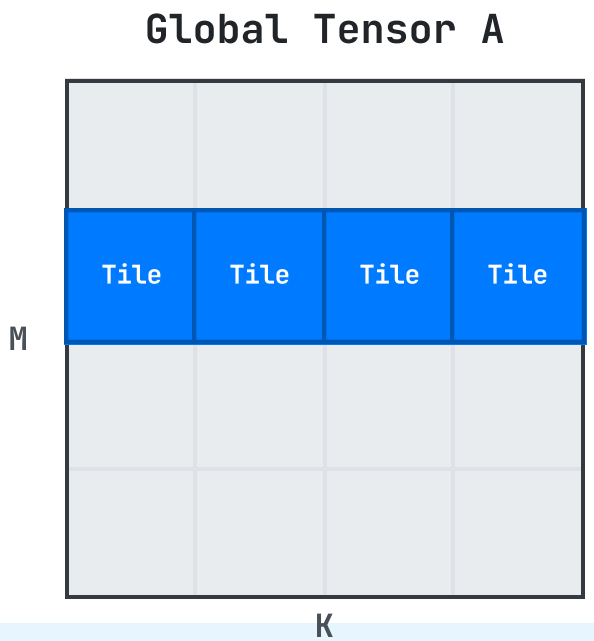
Global Memory, Col Major

We now have handle objects gA, gB, gC that represent the **entire** global matrices.

GEMM in CuTe: CTA Tiling

Partitioning global tensors with a single line of code

```
// 1. Define Tile Shapes
auto bM = make_shape(Int<128>{}, Int<8>{});
auto bN = make_shape(Int<128>{}, Int<8>{});

// 2. Slice Global Tensor A
auto tAgA = local_tile(gA, bM,
                       make_coord(blockIdx.y, _));

// 3. Slice Global Tensor B
auto tBgB = local_tile(gB, bN,
                       make_coord(blockIdx.x, _));
```



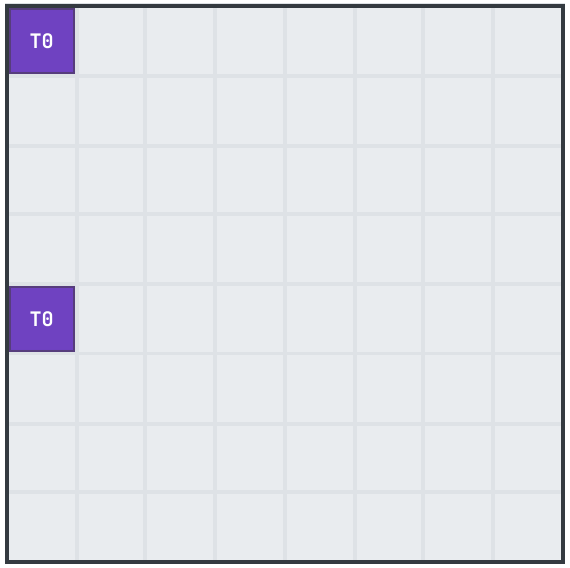
tAgA is now a view into Global Memory containing **only** the row of tiles needed by this CTA.

GEMM in CuTe: Thread Partitioning

Assigning data to threads without manual index arithmetic

```
// 1. Define Thread Layout (e.g., 32 threads)
auto thr_layout = make_layout(
                    make_shape(Int<32>{}, Int<8>{}),
                    make_stride(Int<1>{}, Int<32>{})
);

// 2. Partition the CTA Tile
auto tAgA_frag = local_partition(gA, thr_layout,
                                 threadIdx.x);

// Result: a small local tensor (fragment) for this
thread
```

CTA Tile (Partitioned)



Elements owned by Thread 0

Coordinate Transformation: CuTe automatically calculates the complex strided indices for tAgA_frag.

GEMM in CuTe: The Mainloop

Readable, semantic, and free of index arithmetic

CuTe Kernel

```
// Main Loop over K-tiles
for (int k = 0; k < K_TILES; ++k) {
    // 1. Load Global -> Shared
    copy(tAgA(_,_,k), tAsA);
    copy(tBgB(_,_,k), tBsB);
    cp_async_fence();
    cp_async_wait<0>();
    __syncthreads();

    // 2. Compute (GEMM)
    gemm(tAsA, tBsB, tCrC);
}
```

✓ No Index Math

The loop contains **zero** integer arithmetic.

✓ Semantic Actions

We read "Copy" and "Gemm", not loops and array accesses.

✓ Layout Agnostic

This code works for **any** valid layout of A and B (Row, Col, or Swizzled).

*The complexity hasn't disappeared—it has been encapsulated into the **Layout** objects.*

Tensor Cores: The Compute Atom (MMA)

Hardware Acceleration Wrapped in Layout Algebra

1. Operation (PTX)
Raw instruction wrapper (e.g., mma.m8n8k4)

↓

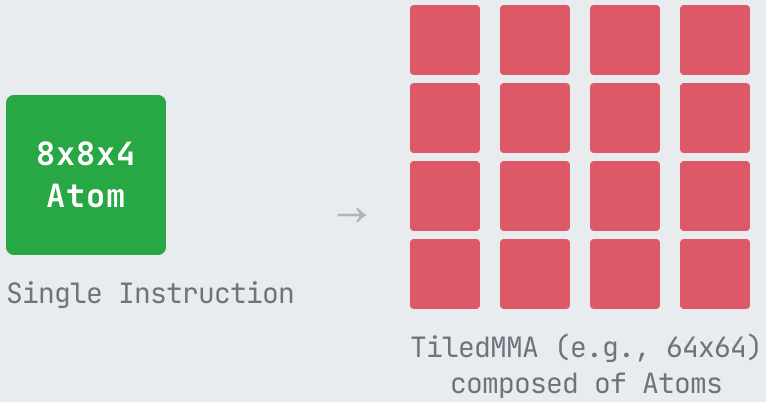
2. Traits (Layouts)
Defines A/B/C layouts for threads & values

↓

3. MMA Atom
Combines Op + Traits into a single unit

↓

4. TiledMMA
Tiles the Atom across threads to cover larger shapes




```
using TiledMMA = decltype(make_tiled_mma(
  MMA_Atom<SM80_16x8x16_F32F16F16F32_TN>{},
  make_layout(Shape<_2,_2,_1>{}) // Tile 2x2 atoms
));
```

Tensor Memory Accelerator: Async Copy

TMA: Moving Tiles without Threads

Manual Thread Copy

Every thread computes its own address, issues a load, and waits.

High Overhead:
Address math + Register pressure.

```
// Manual Index Math
int offset = (by * BM + ty) * N + ...;
if (offset < limit) {
  smem[ty] = gmem[offset];
}
```

TMA Bulk Copy

Hopper +

One instruction copies an entire multi-dimensional tile asynchronously.

Zero Overhead:
Hardware handles addresses.

```
// CuTe Abstraction
auto tma = make_tma_copy(gmem_tensor, ...);
copy(tma, tAg, tAs); // 1 line!
// Layout Algebra generates coords
```

CuTe treats TMA as just another **Copy_Atom**. You define the layout, CuTe generates the descriptor.

The Complete Hierarchy: Atoms to Grid

Modern GPU Decomposition (Hopper+)

Global Tensor

The full matrix in HBM.

`make_tensor`

Layout: (M, K)

→

H100+

Cluster Tile

Group of CTAs sharing SMEM. Multicast & distributed SMEM.

`ClusterLayout`

Layout: (CM, CK)

→

Block Tile

Single CTA slice. Copied via TMA.

`local_tile`
`copy(tma)`

Layout: (BM, BK)

→

Thread Frag

Per-thread partition. Registers.

`local_partition`
`copy`

Layout: (TM, TK)

→

Tensor Core

Hardware instruction consumption.

`gemm(mma)`

Atom: 16x8x16

CuTe's **Layout Algebra** unifies all 5 levels—from HBM to Cluster to Shared to Register to Core.

Navigating the Hierarchy with CuTe

The Verbs: How we move between levels

1. Slicing (Global → Block)

Extract a specific tile from a larger tensor based on a coordinate.

```
auto gmem_tile = local_tile(tensor, tile_shape, blockIdx);
```

2. Partitioning (Block → Thread)

Distribute elements of a tile across threads (View from a single thread).

```
auto thr_frag = local_partition(gmem_tile, t_layout, threadIdx);
```

3. Operation (Thread → MMA)

Execute math on the registers owned by the thread.

```
gemm(tiled_mma, thr_frag_A, thr_frag_B, thr_frag_C);
```

Key Insight:
*Both local_tile and local_partition just return **New Tensors** (Views).*
No data is moved until you call copy() or gemm().

Contrast: Raw C++ vs. CuTe

From Index Arithmetic to Layout Algebra

Raw CUDA/C++

Indexing ✖

Manual integer arithmetic. Fragile and hard to read. `row * stride + col`

Tiling ✖

Implicit in loop bounds. No object represents a "tile". `for (k=0; k<K; k+=BK)`

Refactoring ✖

Changing layouts (e.g., Row to Col major) requires rewriting all index math.

CuTe

Indexing ✔

Logical coordinates. The Layout object handles the math. `tensor(coord)`

Tiling ✔

Explicit Tensor objects created via local_tile.

`auto tile = local_tile(...)`

Refactoring ✔

Layout-agnostic. Just swap the Layout object, the algorithm stays the same.

Static Verification

Catching structural bugs at compile time

The Scenario: Mismatched Copy

```
auto src = make_tensor(ptrA, make_layout(Int<128>{})); // Size 128
auto dst = make_tensor(ptrB, make_layout(Int<64>{}));  // Size 64

copy(src, dst); // Oops! Sizes don't match.
```

❌ Raw C++

```
$ nvcc kernel.cu -o gemm
Build complete.

$ ./gemm
Segmentation fault (core dumped)

# Debugging time: 4 hours
```

✅ CuTe

```
$ nvcc kernel.cu -o gemm
error: static assertion failed: "Dst size must be ≥ Src
size"

# Debugging time: 10 seconds
```

Layout Agnostic Algorithms

Decoupling "What to do" from "Where data lives"

SCENARIO A: TN GEMM

A: Row-Major (M, K)
B: Col-Major (N, K)

SCENARIO B: NN GEMM

A: Col-Major (M, K)
B: Col-Major (K, N)

THE KERNEL CODE (UNCHANGED)

```
template <class TA, class TB, class TC>
__global__ void gemm_kernel(TA tA, TB tB, TC tC) {
    copy(tAgA, tAsA);
    gemm(tAsA, tBsB, tCrC);
}
```

Example:
Transpose via Layout

```
// A: (M, N) Col-Major
// B: (M, N) Row-Major
copy(A, B); // <--- This IS a Transpose!
```

*We write the algorithm **once**. The Layouts determine the data movement.*

Beyond GEMM: General Applicability

Layout Algebra is a general theory of multi-dimensional arrays

Convolutions

Implicit im2col

Convolution is just a coordinate transformation. We can view a 4D image tensor as a 2D matrix without copying data.

```
// (N,H,W,C) → (M, K)
auto matrix_view =
  logical_divide(image, ...);
```

Tensor Contractions

Arbitrary Dimensions

GEMM is just a contraction of rank-2 tensors. CuTe handles rank-N contractions by grouping modes.

```
// T(i,j,k,l) → T((i,j), (k,l))
auto grouped =
  group_modes<0,2>(tensor);
```

Memory Swizzling

Bank Conflict Avoidance

Swizzling functions (XOR patterns) are just "Layout Atoms" that can be composed with any other layout.

```
// XOR bits 2,3 with 0,1
auto swizzle =
  composition(layout,
Swizzle<2,2,3>{});
```

If you can define the index math, you can define a **Layout**.

Key Takeaways

Why CuTe changes high-performance programming

1 Layouts are Functions

They map logical coordinates to physical offsets. This abstraction completely decouples **what** data you want from **where** it is stored.

2 Algebra > Arithmetic

Stop writing row * stride + col. Use algebraic operations like composition, product, and division to manipulate data structures.

3 Static Verification

Layouts carry shape and stride information in their **type**. The compiler catches structural errors (like shape mismatches) when you compile the code.

4 Layout Agnostic

Write your algorithm once. It will work for Row-Major, Column-Major, or complex Swizzled layouts without changing a single line of kernel code.

Resources and Next Steps

Where to go from here

Official CuTe Documentation

The definitive guide to Layouts, Tensors, and MMA Atoms.

github.com/NVIDIA/cutlass/blob/main/media/docs/cute/00_quickstart.md

CuTe GEMM Tutorial

A step-by-step walkthrough of building a high-performance GEMM kernel.

github.com/NVIDIA/cutlass/blob/main/media/docs/cute/0x_gemm_tutorial.md

GPU Mode Lecture 57: Cris Cecka

Cris's Lecture on CuTe Layouts has a lot of the rationale behind CuTe Layouts.

www.youtube.com/watch?v=ufa4pmBOBT8

Colfax Research

Categorical Foundations for CuTe Layouts

research.colfax-intl.com

Thank You!

Questions?